

Evaluating the model

- we split the feature matrix into training and testing to make the model generalize well

Dataset:

size	price
2104	400
1600	330
2400	369
1416	232
3000	540
1985	300
1534	315
1427	199
1380	212
1494	243

70%

30%

training
set

$m_{train} = 7$

test set

$m_{test} = 3$

(x^T, y^T)

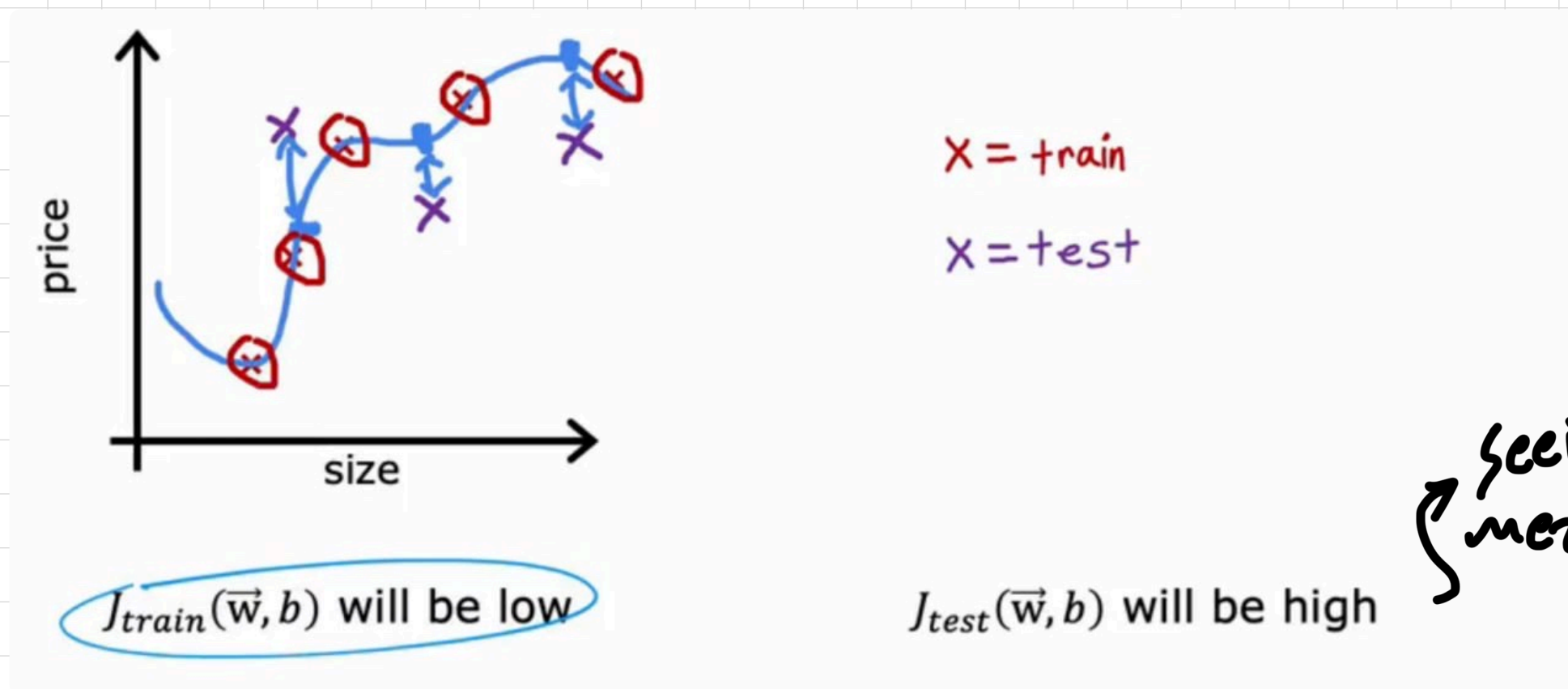
$x^{(m_{train})}, y^{(m_{train})}$

$x^{(m_{test})}, y^{(m_{test})}$

Train/test Procedure for LR with MSE

$$J_{\text{train}}(\vec{w}, b) = \frac{1}{2n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} (\hat{y}_i - y_i)^2 + \frac{\lambda}{2n_{\text{train}}} \sum_{j=1}^n w_j^2 \quad \text{train}$$

$$J_{\text{test}}(\vec{w}, b) = \frac{1}{2n} \sum_{i=1}^{n_{\text{test}}} (y_{i_{\text{test}}} - \hat{y}_{i_{\text{test}}})^2 \quad \text{test}$$



seeing J_{test} high on this model means it's not good at generalizing

Model selection

$d=1$ $f_{\vec{w},b}(\vec{x}) = w_1 x + b$ linear model $\rightarrow J_{\text{test}}(w^1, b^1)$

$d=2$ $f_{\vec{w},b}(\vec{x}) = w_1 x + w_2 x^2 + b$

$d=3$ $f_{\vec{w},b}(\vec{x}) = w_1 x + w_2 x^2 + w_3 x^3 + b$

$d=10$ $f_{\vec{w},b}(\vec{x}) = w_1 x + w_2 x^2 + \dots + w_{10} x^{10} + b$

$\rightarrow J_{\text{test}}(w^2, b^2)$

Non-linear w^3, b^3

w^{10}, b^{10}

- Because we looked at the test set many times to choose the best d

- This will make the performance look better than it truly does

↓ Proper model selection

We look at all costs and we take the lowest value

e.g. $J_{\text{test}}(w^5, b^5)$ lowest
 $d=5$

- If we use the test set to pick d then the report is too optimistic

Training / cross validation / testing set

Dataset:

size	price
2104	400
1600	330
2400	369
1416	232
3000	540
1985	300
1534	315
1427	199
1380	212
1494	243

60% training set $(x_{\text{train}}, y_{\text{train}})$

20% cross validation

$(x_{\text{cv}}, y_{\text{cv}})$

20% test set

$(x_{\text{test}}, y_{\text{test}})$

1- Training set (60%)

Summary

- Used to fit the model
- The model learns the relationship between house size (input) and Price (output)

2- Cross Validation (20%)

- used to choose the model and hyperparameters e.g degree....
- If $d=5$ gives lowest error we on CV we choose it

3- Test set (20%)

- used at the end to see the final performance
- like unseen data
- If we choose d using CV, then we check test set we get realistic estimate of how the model generalizes to unseen data

Training/cross validation/test set

Training error: $J_{train}(\vec{w}, b) = \frac{1}{2m_{train}} \left[\sum_{i=1}^{m_{train}} (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 \right]$

Cross validation error: $J_{cv}(\vec{w}, b) = \frac{1}{2m_{cv}} \left[\sum_{i=1}^{m_{cv}} (f_{\vec{w}, b}(\vec{x}_{cv}^{(i)}) - y_{cv}^{(i)})^2 \right]$ (validation or dev error)

Test error: $J_{test}(\vec{w}, b) = \frac{1}{2m_{test}} \left[\sum_{i=1}^{m_{test}} (f_{\vec{w}, b}(\vec{x}_{test}^{(i)}) - y_{test}^{(i)})^2 \right]$

Model selection

$d=1$	1. $f_{\vec{w},b}(\vec{X}) = w_1x + b$	$w^{<1>, b^{<1>}} \rightarrow$	<u>$J_{cv}(w^{<1>, b^{<1>})$</u>
$d=2$	2. $f_{\vec{w},b}(\vec{X}) = w_1x + w_2x^2 + b$	$\rightarrow$	<u>$J_{cv}(w^{<2>, b^{<2>})$</u>
$d=3$	3. $f_{\vec{w},b}(\vec{X}) = w_1x + w_2x^2 + w_3x^3 + b$		$\vdots$
$\vdots$	$\vdots$		
$d=10$	10. $f_{\vec{w},b}(\vec{X}) = w_1x + w_2x^2 + \dots + w_{10}x^{10} + b$		<u>$J_{cv}(w^{<10>, b^{<10>})$</u>

$\rightarrow$ Pick $w_1x + \dots + w_4x^4 + b$ ($J_{cv}(w^{<4>, b^{<4>})$) **lowest**

Estimate generalization error using test the set: $J_{test}(w^{<4>, b^{<4>})$

- **CV**: Choosing the model
- **Test**: final evaluation
- **train**: train the model

$\downarrow$

• After we choose $J_{cv}(w^*, b^*)$ we check its performance on $J_{test}(w^*, b^*)$

EXAMPLE on one feature

X_{train} shape $\rightarrow (50, 1)$

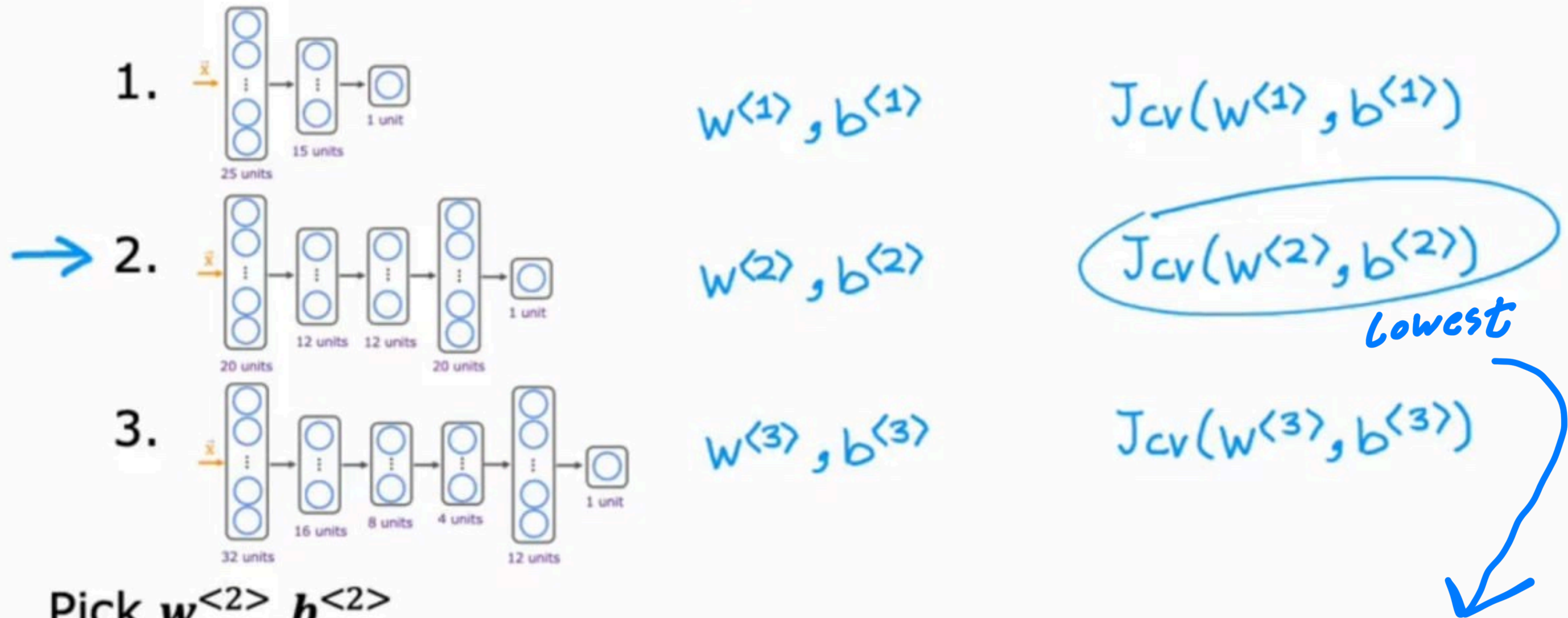
✓ Applying Polynomial Feature using PolynomialFeatures from sklearn

$$X_{\text{train_Poly}} = (50, 2)$$
[illegible]

50 ex

model: $\hat{y} = w_0 + w_1 x_1 + w_2 x^2$

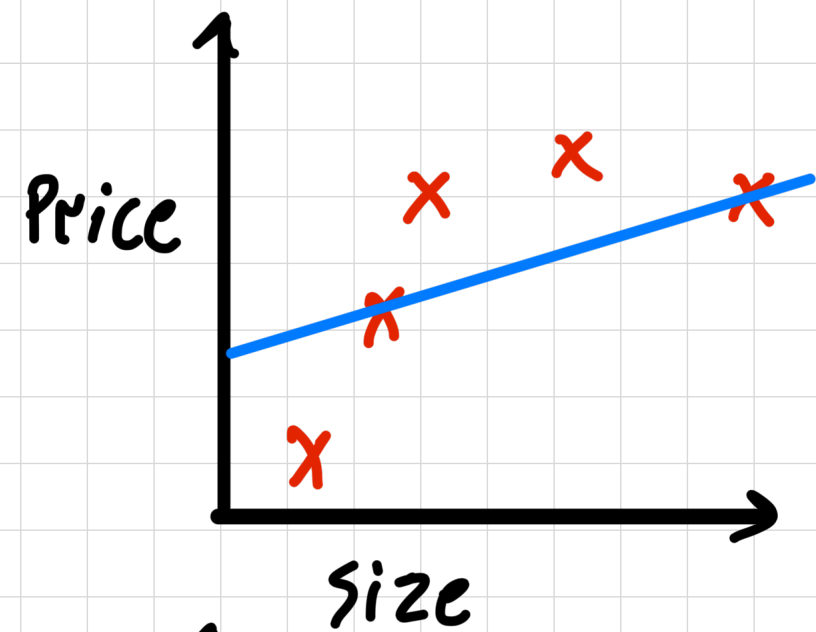
Model selection – choosing a neural network architecture



Pick $w^{(2)}, b^{(2)}$

Estimate generalization error using the test set: $J_{test}(w^{(2)}, b^{(2)})$

Diagnosing bias and Variance

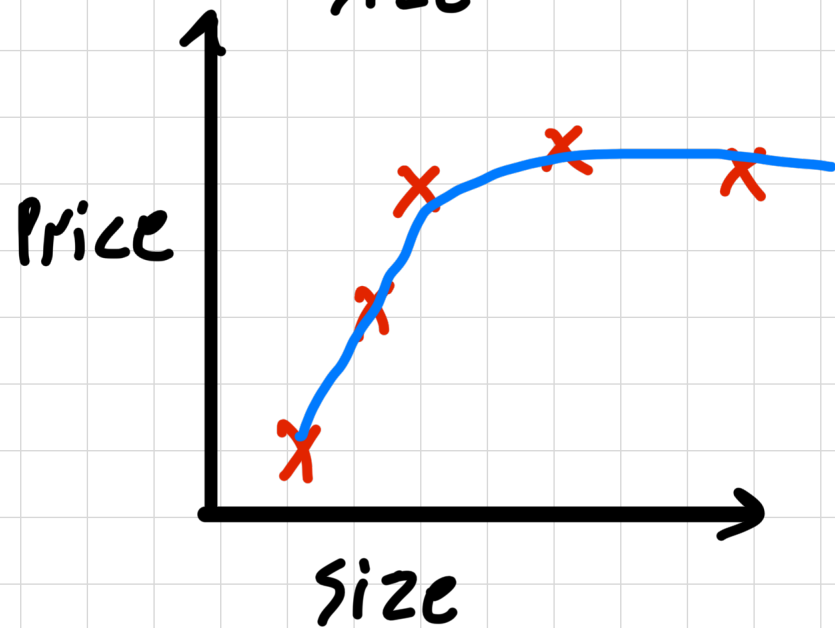


$$\hat{y}(x) = w_1 x + b \quad \text{Linear model}$$

$d=1$ High bias (under fit)

σ_{train} is High
 σ_{CV} is High

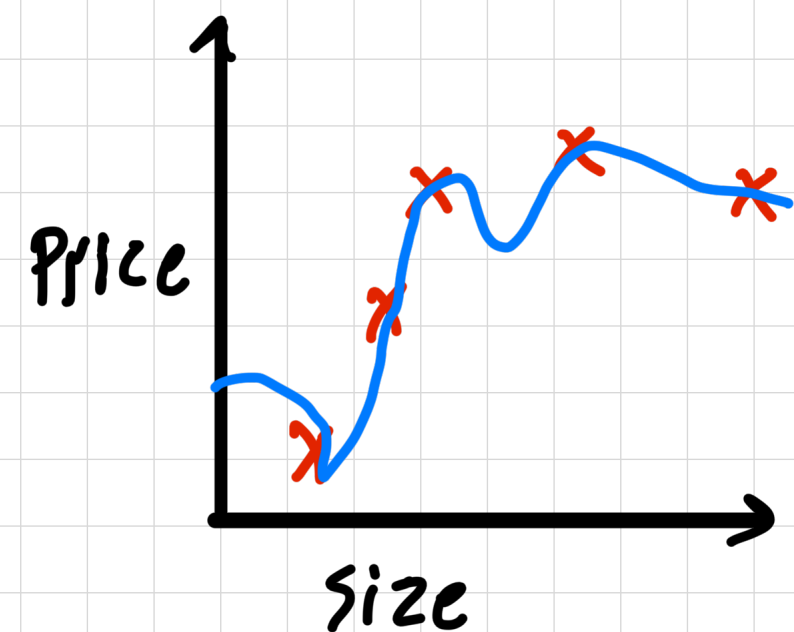
$$\sigma_{\text{CV}} \approx \sigma_{\text{train}}$$



$$\hat{y}(x) = w_1 x + w_2 x^2 + b \quad \text{Non-linear model}$$

$d=2$ Good fit

σ_{train} is low
 σ_{train} is low
 σ_{CV} is low ✓



$$\hat{y}(x) = w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4 + b$$

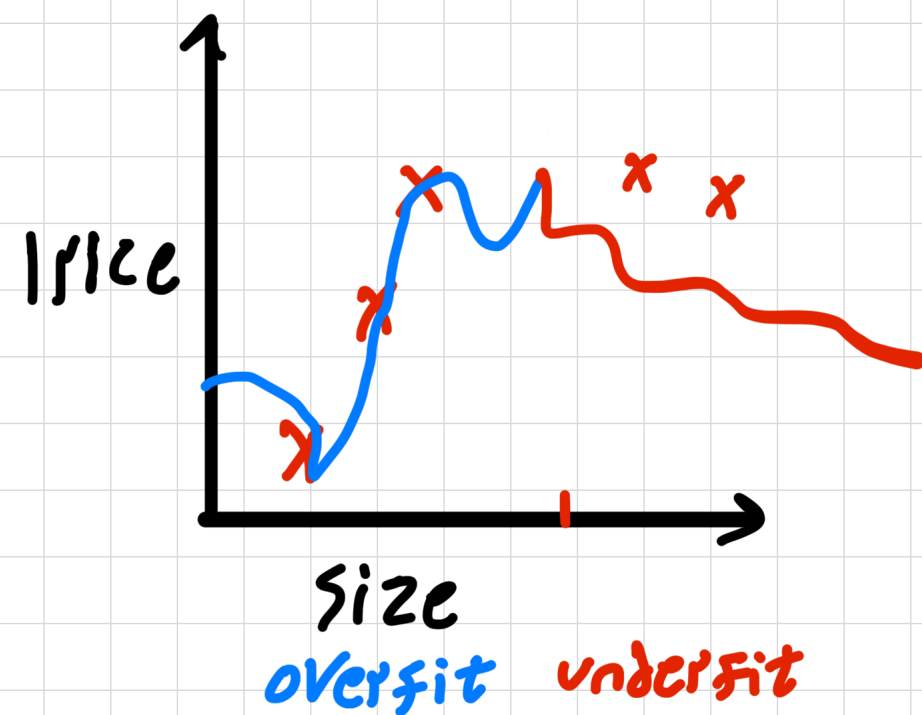
$d=4$ High Variance (over fit)

σ_{train} is low
 σ_{CV} is High

$$\sigma_{\text{CV}} \gg \sigma_{\text{train}}$$

High bias And High Variance at the same time

• sometimes happens in NN

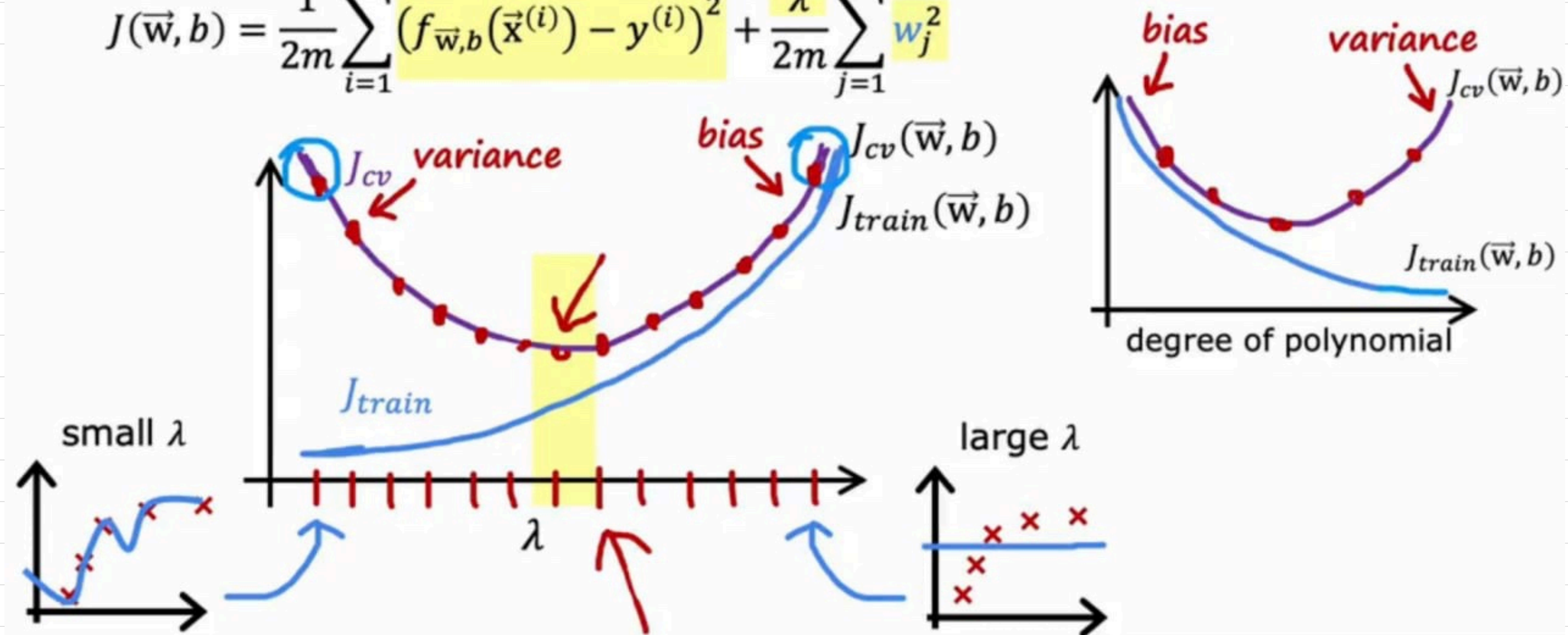


Regularization and bias/variance

- Regularization controls model complexity by penalizing large weights

Bias and variance as a function of regularization parameter λ

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$



Debugging a Learning algorithm

Regularized LR

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^i - y^i)^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

If we have large errors in predictions.

Solution

- 1- Get more training samples Fixes High Variance (overfitting)
- 2- Try smaller sets of features $x, x^2, \cancel{x^3}, \cancel{x^4}$ fixes High Variance
- 3- Try getting additional features Fixes High Bias (underfitting)
- 4- Adding Polynomial features Fixes High Bias
- 5- Decreasing λ Fixes High bias
- 6- Increasing λ Fixes High Variance

The bias variance tradeoff

$$\underline{f_{\vec{w},b}(x) = w_1x + b}$$

Simple model

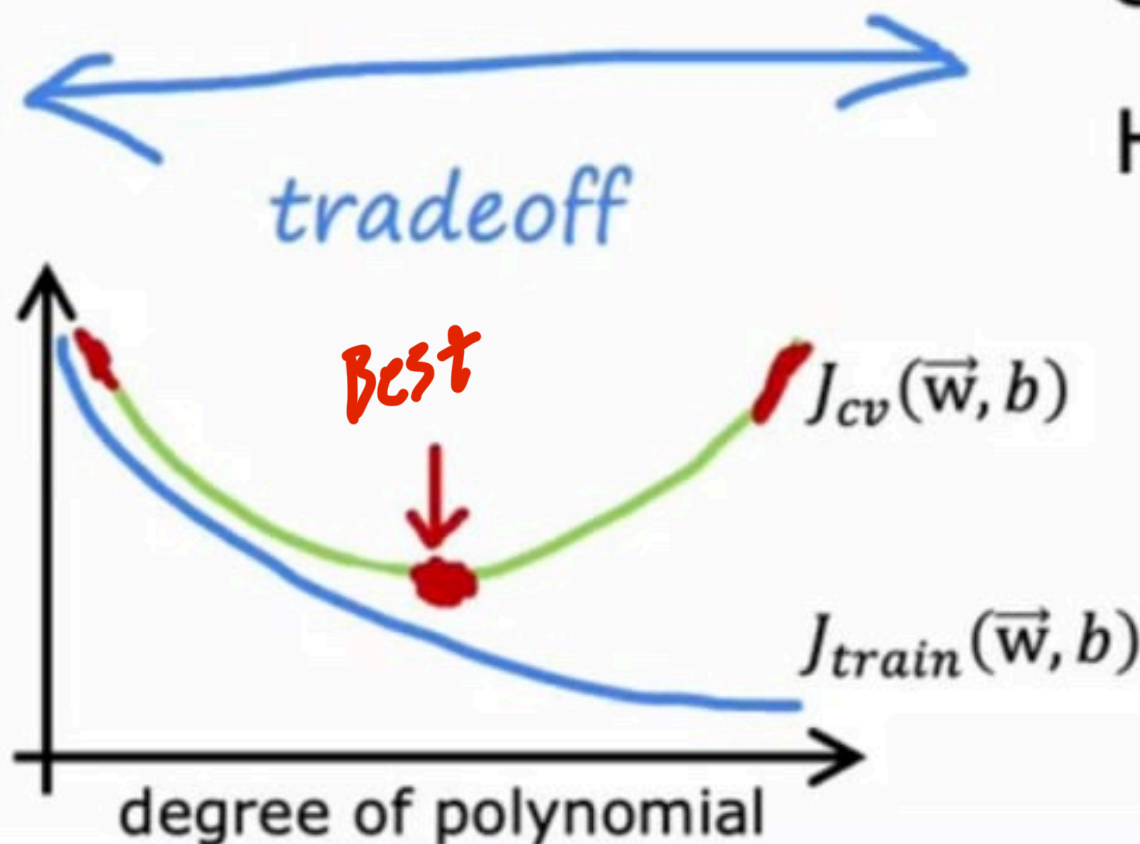
High bias

$$\underline{f_{\vec{w},b}(x) = w_1x + w_2x^2 + b}$$

$$\underline{f_{\vec{w},b}(x) = w_1x + w_2x^2 + w_3x^3 + w_4x^4 + b}$$

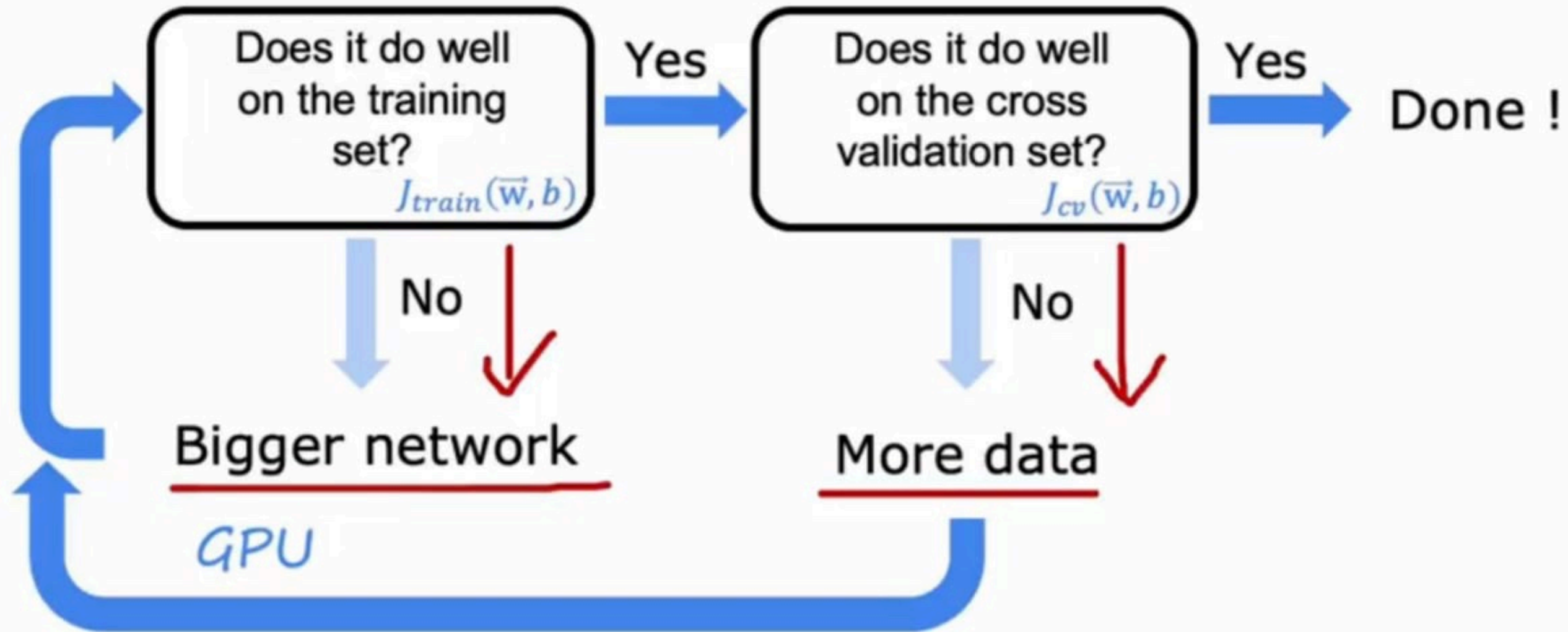
Complex model

High variance



NN and bias/variance

Large neural networks are low bias machines



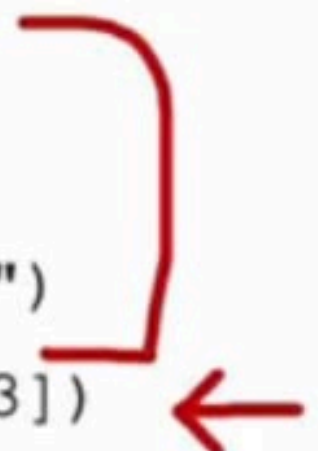
Neural network regularization

$$\underline{J(\mathbf{W}, \mathbf{B})} = \frac{1}{m} \sum_{i=1}^m \underbrace{L(f(\vec{x}^{(i)}), y^{(i)})}_{\text{loss}} + \frac{\lambda}{2m} \sum_{\text{all weights } \mathbf{W}} (w^2)$$

b

Unregularized MNIST model

```
layer_1 = Dense(units=25, activation="relu")
layer_2 = Dense(units=15, activation="relu")
layer_3 = Dense(units=1, activation="sigmoid")
model = Sequential([layer_1, layer_2, layer_3])
```



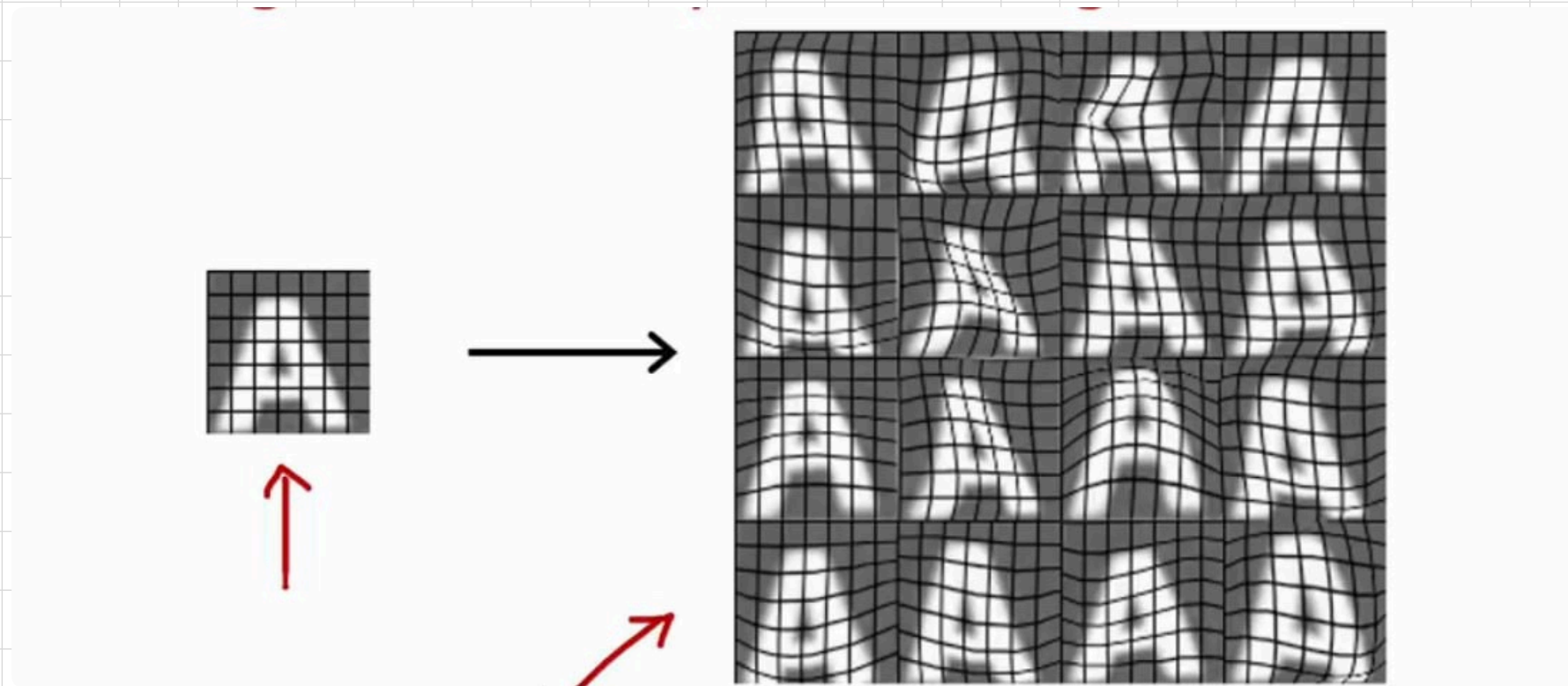
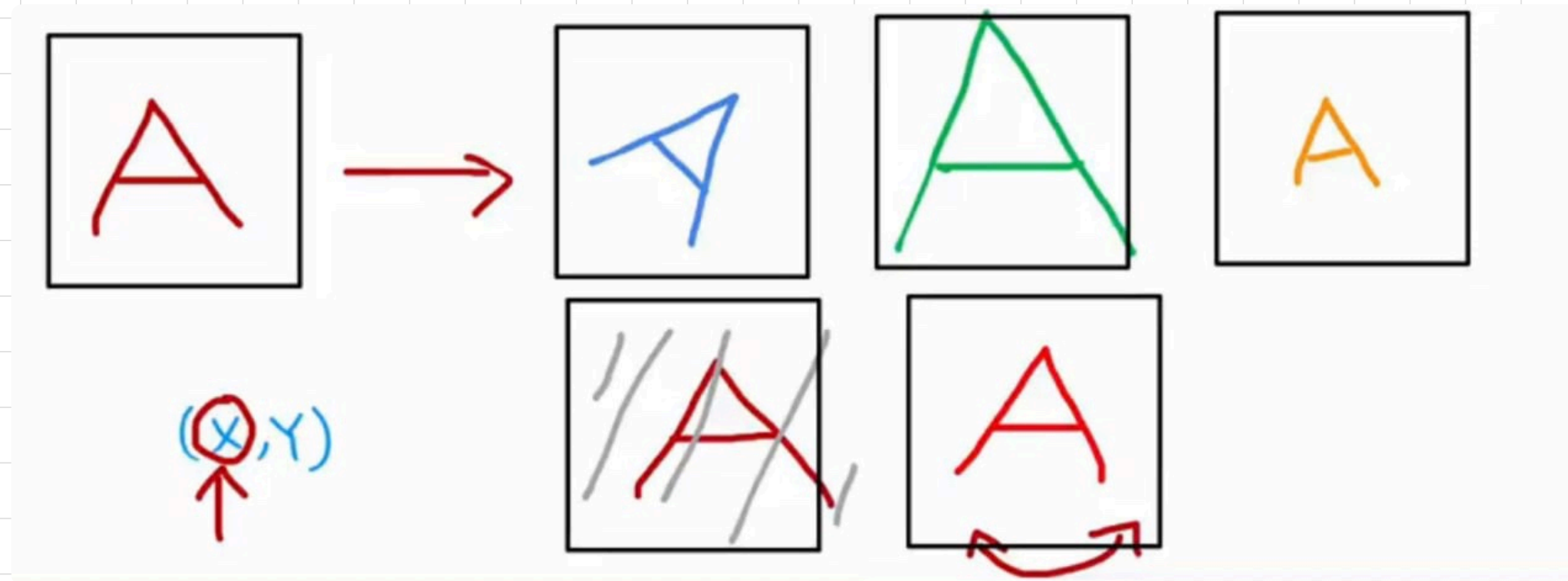
Regularized MNIST model

```
layer_1 = Dense(units=25, activation="relu", kernel_regularizer=L2(0.01))
layer_2 = Dense(units=15, activation="relu", kernel_regularizer=L2(0.01))
layer_3 = Dense(units=1, activation="sigmoid", kernel_regularizer=L2(0.01))
model = Sequential([layer_1, layer_2, layer_3])
```

λ
↓

Dat augmentation

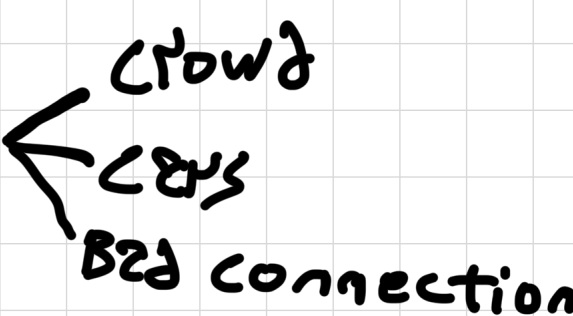
modifying an existing training samples to create a new training ex



more data the algorithm can learn from distorted images

Data augmentation for speech

To apply data augmentation on speech we add noise in background



- crowd
- cars
- B2B connection

Dataset Synthesis

uses artificial data inputs to create a new training ex

Artificial data synthesis for photo OCR



Real data



Synthetic data

Transfer learning

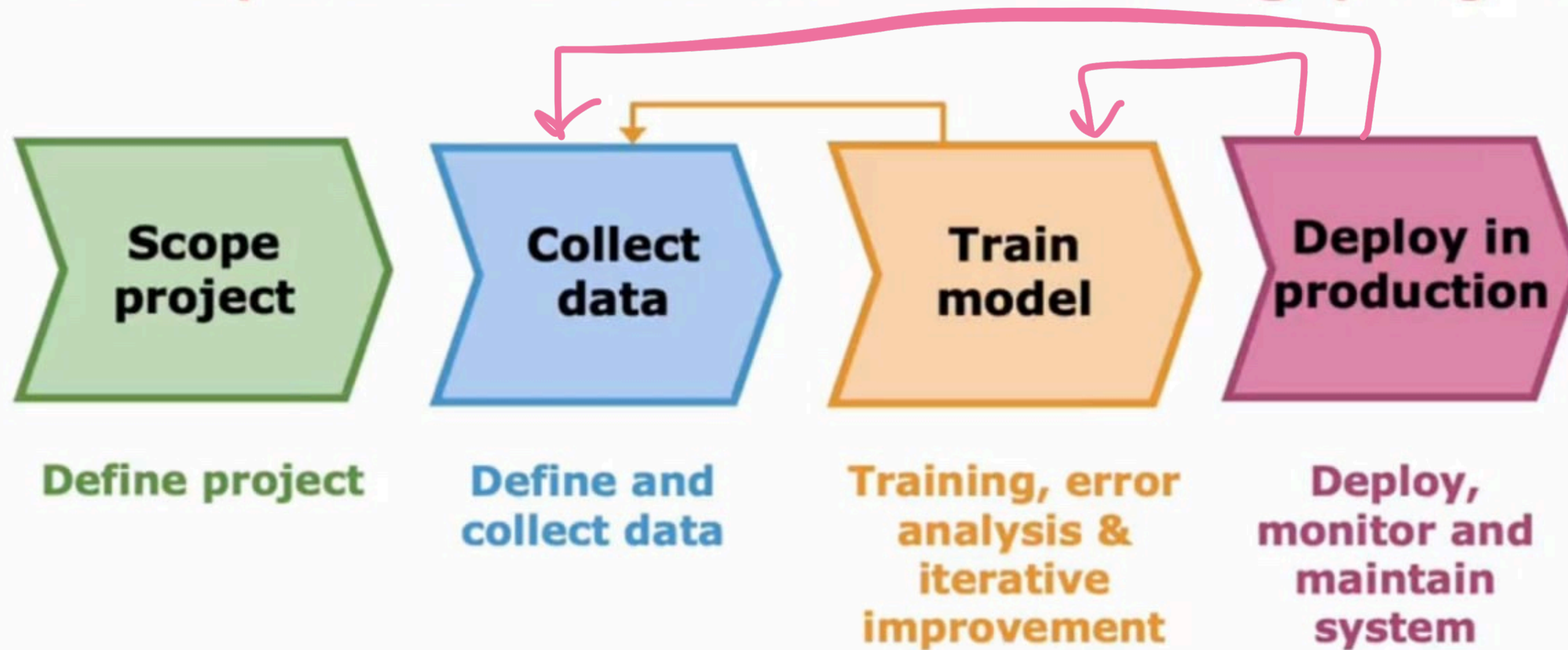
Instead of training model from scratch, starts with a pretrained model has been trained on large dataset, then fine-tune based on the task

Two main steps for transfer learning

- 1- Download NN Parameters **Pretrained** on large dataset **with same input type** as our task
- 2- **Train** the NN on **our data** (fine-tuning)

Machine Learning Process

Full cycle of a machine learning project



Precision/recall

$y = 1$ in presence of rare class we want to detect.

Actual Class		1	0
Predicted Class	1	True positive 15	False positive 5
	0	False negative 10	True negative 70
		↓ 25	↓ 75

Precision:

(of all patients where we predicted $y = 1$, what fraction actually have the rare disease?)

$$\frac{\text{True positives}}{\text{\#predicted positive}} = \frac{\text{True positives}}{\text{True pos} + \text{False pos}} = \frac{15}{15 + 5} = 0.75$$

Recall:

(of all patients that actually have the rare disease, what fraction did we correctly detect as having it?)

$$\frac{\text{True positives}}{\text{\#actual positive}} = \frac{\text{True positives}}{\text{True pos} + \text{False neg}} = \frac{15}{15 + 10} = 0.6$$

```
print("y=0")
```

Precision = $\frac{TP}{TP + FP}$ when the model predict $\hat{y} = 1 \rightarrow$ patient has disease ^{↑ confident} How often this is correct?

• Predict 1 if $F_{w,b}(\vec{x}) \geq 0,7$ or $0,9$: means don't Predict 1 unless you're 90% sure

• Precision ↑↑

• Recall ↓↓

Recall = $\frac{TP}{TP + FN}$ out of all people who really have the disease How many did the model find?

• If we don't want to miss many cases of rare disease

$F_{w,b}(\vec{x}) \geq 0,3, 0,1$, possibly missing 2 case

• Precision ↓↓

• Recall ↑↑

$$F_1 \text{ score} = \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \times 2 \quad \text{Helps Comparing between models}$$

F1 score

How to compare precision/recall numbers?

	Precision (P)	Recall (R)	Average	F ₁ score
Algorithm 1	0.5	0.4	0.45	0.444
Algorithm 2	0.7	0.1	0.4	0.175
Algorithm 3	0.02	1.0	0.501	0.0392

`print("y=1")`

~~$$\text{Average} = \frac{P+R}{2}$$~~

$$F_1 \text{ score} = \frac{1}{\frac{1}{2} \left(\frac{1}{P} + \frac{1}{R} \right)} = 2 \frac{PR}{P+R}$$

Harmonic mean

